

# 本项目开发画像与数字孪生成熟度评估

基于 ti-campus-template 项目代码与公开权威资料的独立分析

版本：2026-08-21 | 统计口径：正文非空白字符约 3000—4000

## 摘要

本报告以 `ti-campus-template` 当前代码为对象，结合数字孪生标准、行业框架和近期研究，回答两个问题：这个项目目前属于什么类型的数字孪生前端；哪些能力已经形成工程基础，哪些仍停留在三维可视化或项目化实现阶段。结论是：项目已经越过“模型浏览器”阶段，形成了围绕校园、建筑、楼层、房间和设备的实时业务编排框架，具备数据接入、层级联动、状态表达和有限交互控制能力；但距离可验证、可预测、可闭环控制的高成熟度数字孪生仍有明显距离。综合评定约为五级中的 2.5 至 3 级，即“可观测、可交互的业务孪生”，尚不是高可信仿真或自主决策系统。

## 一、评估口径：数字孪生不等于三维大屏

NIST 将数字孪生概括为现实实体的电子表征，并强调对实体状态与状态转移进行观察、评估、模拟或控制。ISO 23247 则把术语、参考架构、数字表达和信息交换分开定义。两者共同提示：三维模型只是表达层，真正决定孪生成熟度的是实体身份、数据同步、行为模型、时间一致性、验证与治理。本项目有园区模型和大量 3D 效果，但评估不能以“画面是否炫酷”为核心，而应看物理对象、业务数据和场景行为是否形成可追踪的对应关系。

本报告采用六个维度：空间表达、实体与数据映射、实时同步、业务编排、仿真预测、治理可信。五级标尺为：一级静态可视化；二级数据映射；三级实时监测与交互；四级预测和方案推演；五级受控自治。评分是面向当前仓库的工程判断，不代表完整部署环境或后端平台能力。

## 二、当前工程画像

项目技术栈为 Vue 3.5、TypeScript、Vite、Pinia、ThingJS 2.0 及优锑场景仓库组件，前端约有一百个 TypeScript 文件和五十余个 Vue 文件。`src/components/Scene.vue` 负责创建三维容器、加载主园区与效果包、初始化孪生点位、告警和性能 WebSocket，并衔接建筑与楼层顶牌。其体量超过六百行，仍承担较多总装职责，但核心业务已向独立管理器迁移。

业务主线不是 README 所写的 `BaseStory/src/stories`，而是 `BusinessInitManager + BaseBusinessLogic + registry + Manager`。注册表包含综合态势、全景空间、宿舍管理、能源管理四类一级业务。`BusinessInitManager` 统一处理业务切换、回到 Campus、等待 mounted、过期序列回收、超时与 AbortSignal；`BaseBusinessLogic` 统一处理 Campus、Building、Floor 进入离开以及大屏组件同步。这说明项目已经意识到 3D 对象生命周期与 Vue 组件生命周期不同，且开始通过状态机式编排降低残留对象、重复监听和异步竞态。

数据侧已经形成 HTTP、Mock、WebSocket 和配置注册链路。`fetchApiData.ts` 支持接口轮询、更新回调和停止轮询；告警和性能数据通过 WebSocket 更新；房间、人员、监控、工单、停车、建筑和能耗均有接口或 Mock。OpenSpec 中明确“接口文档—Mock—注册—变更记录—代码调用”的顺序，这是 Agent 协作所需的可检索约束。不过当前自动化测试主要集中在语音令牌服务，三维业务、视觉状态和资源清理缺少等比例的回归验证。

业务能力方面，综合态势已实现停车分布、监控分布和建筑顶牌联动；宿舍管理包含床位 Box、HTMLMarker、入住状态、楼层切换以及能耗子业务；能源可视化存在液体体块、图表 Marker 与逐帧更新；全景空间和漫游支持路线、业务主题切换、图层动作、语音解说与音频缓存。楼层管理器负责

建筑展开、楼层标识和显隐。项目还显式处理相机缩放软限位、近裁剪面、Z-fighting、Marker 离屏挂载、事件 tag 与 destroy，这些都是普通二维前端很少遇到、但 3D 工程稳定性高度依赖的细节。

### 三、六维成熟度判断

空间表达可评 3.5 分。Campus/Building/Floor/Room 层级清楚，支持建筑、楼层、设备、Marker、体块和视角联动，坐标和父子关系已成为代码的一等概念。短板是空间语义仍分散在对象名称、效果标签、接口字段和全局变量中，缺少统一空间实体模型与坐标契约。

实体与数据映射可评 3 分。项目可以通过 UUID、场景对象 ID、建筑和楼层信息关联业务数据，具备房间、床位、监控和停车位的多类映射。问题是字段语义、唯一标识、时间戳、质量标记和数据血缘没有统一模型；部分逻辑依赖名称匹配或全局实例，跨项目迁移时容易失效。

实时同步可评 3 分。HTTP 轮询与 WebSocket 已覆盖告警、性能及业务刷新，场景层级变化也会驱动数据与可视对象重建。但仓库内看不到端到端的时钟同步、乱序去重、断线补偿、数据新鲜度展示和离线降级统一策略。实时“收到数据”不等于可信“同步现实”。

业务编排可评 3.5 分。统一注册、生命周期、切换序列、超时、取消和安全销毁是明显优势，漫游还能够跨业务和图层执行动作。短板是 `Scene.vue`、`BuildingManager`、`BedManager` 等文件仍较大，少量历史 hook 与新 Manager 并存；README 架构说明与实际代码漂移，会导致新人和 Agent 选择错误入口。

仿真预测可评 1.5 分。现有路线移动、楼层展开、液体体块和相机动画属于表现或流程仿真，尚未看到基于物理、统计或运筹模型的“如果—那么”推演，也缺少模型校准、误差区间与预测回测。因此不应把动画播放宣传为预测型数字孪生。

治理可信可评 2 分。项目有规格、接口文档、Mock、构建和部分清理规范，但 3D 回归测试、性能预算、安全边界、数据质量、审计日志、权限分级和变更影响分析仍不足。NIST 2026 年研讨报告将互操作、验证确认与不确定性量化、网络安全和人才能力列为持续障碍，本项目基本对应这些行业共性问题。

### 四、客观优势与主要债务

最大优势是“业务状态—层级—三维对象”已形成真实耦合，不是简单贴图大屏；第二，业务切换和销毁被当作核心能力；第三，接口文档和 Mock 为离线开发与 Agent 检索提供了基础；第四，校园场景覆盖停车、监控、宿舍、能耗和漫游，足以构建有代表性的评测集。

主要债务有四类。其一，架构文档漂移，README 宣称的 Story 模式与代码主线不一致。其二，自动化验证偏弱，尤其缺少重复进入退出、切层、Marker 残留、WebGL 内存和视觉差异测试。其三，全局 `window.\*Ins` 方便集成，却增加隐式依赖和并发切换风险。其四，能源管理主 Manager 中大量旧实现被注释保留，说明能力边界尚在调整，Agent 若只做文本匹配容易复活废弃路径。

## 五、结论与优先级

未来不应优先增加更多视觉特效，而应先补齐“可证明正确”。第一优先是建立场景能力清单和单一架构事实源，明确 ThingJS API、自定义业务逻辑、对象 parent、坐标系、事件 tag 和销毁责任。第二优先是把 Campus—Building—Floor、业务切换、数据刷新、漫游中断建立为可重复测试。第三优先是为数据增加来源、时间、质量和新鲜度语义。第四优先才是引入预测模型或运行时 Agent，并先限制为解释、检索和建议，不直接控制物理设备。

从商业视角看，项目已经具备展示和运营辅助价值，也具备让编码 Agent 提效的结构基础；但任何“自主孪生”“智能决策”表述都应附带数据质量、验证结果和人工审批边界。成熟度提升的关键不是模型更大，而是实体、数据、行为和证据链更完整。

## 参考资料

1. NIST IR 8620, Digital Twins Workshops Summary Report, 2026:  
<https://www.nist.gov/publications/digital-twins-workshops-summary-report>
2. NIST IR 8356, Security and Trust Considerations for Digital Twin Technology, 2025:  
<https://www.nist.gov/publications/security-and-trust-considerations-digital-twin-technology-digital-twin-technology>
3. ISO 23247-1:2021: <https://www.iso.org/standard/75066.html>
4. Digital Twin Consortium, Spatially Intelligent Digital Twin Capabilities, 2025:  
<https://www.digitaltwinconsortium.org/press-room/03-27-25/>
5. ThingJS 官方 API: <https://support.thingjs.com/api>